# Práctica 2 — Multiplicación rápida de enteros: Algoritmo de Karatsuba

Análisis y Diseño de Algoritmos — Universidad EAFIT

Santiago Díaz · Juan Diego Martínez Jaramillo · Samuel Molina Garcés

Noviembre de 2026 · Repositorio: `github.com/Samuel07M/ADA_P2_Diaz_Martinez_Molina`

Implementamos en C++ la multiplicación clásica $\Theta(n^2)$ y la de Karatsuba $\Theta(n^{\log_2 3})$ sobre números en base 10 guardados como cadenas. Karatsuba pasa de 4 a 3 productos recursivos, y eso baja el exponente de 2 a $\log_2 3 \approx 1{,}585$. Con caso base de 64 dígitos, Karatsuba supera al clásico desde $n = 128$ y a 4096 dígitos es 4,4 veces más rápido. Con recursión hasta 3 dígitos (Karatsuba "puro") el cruce no aparece en el rango 2–4096.

## 1. Deducción de la identidad de Karatsuba

Partimos $x$ e $y$ de $n$ dígitos en la posición $m = n/2$, con base $B = 10$: $x = x_1 B^m + x_0$ y $y = y_1 B^m + y_0$, donde $x_1, y_1$ son las mitades altas y $x_0, y_0$ las bajas. Al multiplicar y agrupar por potencias de $B$:

$$x \cdot y = x_1 y_1 \, B^{2m} + (x_1 y_0 + x_0 y_1) \, B^m + x_0 y_0.$$

Así hacen falta **cuatro** productos de mitades. Pero del término central solo necesitamos la *suma* $x_1 y_0 + x_0 y_1$, no cada producto. Definimos tres productos:

$$P_1 = x_1 y_1, \qquad P_2 = x_0 y_0, \qquad P_3 = (x_1 + x_0)(y_1 + y_0).$$

Al expandir $P_3$ aparecen los cuatro productos cruzados:

$$P_3 = x_1 y_1 + x_1 y_0 + x_0 y_1 + x_0 y_0 = P_1 + (x_1 y_0 + x_0 y_1) + P_2 \implies x_1 y_0 + x_0 y_1 = P_3 - P_1 - P_2,$$

y la identidad queda

$$\boxed{x \cdot y = P_1 \, B^{2m} + (P_3 - P_1 - P_2) \, B^m + P_2}$$

**Por qué tres y no cuatro.** Cambiamos una multiplicación por dos sumas ($x_1 + x_0$, $y_1 + y_0$) y dos restas, que cuestan $\Theta(n)$, mientras que cada multiplicación es una llamada recursiva completa. Ahorrar un producto en *cada* nivel cambia el orden de crecimiento, no solo una constante (secciones 2 y 3). El precio es que $x_1 + x_0$ puede tener $m + 1$ dígitos.

**Orden de la resta.** Con dígitos sin signo calculamos $(P_3 - P_1) - P_2$: ninguna resta da negativo, porque $P_3 - P_1 = P_2 + x_1 y_0 + x_0 y_1 \geq P_2$.

**Ejemplo.** $1234 \times 5678$ con $m = 2$: $P_1 = 12 \cdot 56 = 672$, $P_2 = 34 \cdot 78 = 2652$, $P_3 = 46 \cdot 134 = 6164$. El centro es $6164 - 672 - 2652 = 2840$ y $672 \cdot 10^4 + 2840 \cdot 10^2 + 2652 = 7\,006\,652$.

## 2. Recurrencia por el Teorema Maestro

Cada llamada hace tres llamadas sobre mitades y un trabajo lineal (partir, sumar, restar, desplazar, recomponer):

$$T(n) = 3\,T(n/2) + \Theta(n), \qquad a = 3,\ b = 2,\ f(n) = \Theta(n),\ n^{\log_b a} = n^{\log_2 3} \approx n^{1{,}585}.$$

Como $\log_2 3 > 1$, $f(n)$ crece polinomialmente más despacio que $n^{\log_b a}$: existe $\varepsilon > 0$ (por ejemplo $\varepsilon = 0{,}5$) con $f(n) = n = O(n^{\log_2 3 - \varepsilon})$. Es el **caso 1**: domina el costo de las hojas y

$$T(n) = \Theta\left(n^{\log_2 3}\right) \approx \Theta\left(n^{1{,}585}\right).$$

**Lo que aporta:** la clasificación rápida comparando $f(n)$ con $n^{\log_b a}$. Como contraste, el esquema de cuatro productos tiene $a = 4$, $n^{\log_2 4} = n^2$, y da $\Theta(n^2)$: no mejora al clásico.

## 3. Recurrencia por árbol de recurrencia

Sea $c\,n$ el trabajo lineal de una llamada de tamaño $n$, con $n$ potencia de 2 (lo garantiza el relleno inicial).

| Nivel $i$ | Subproblemas | Tamaño de cada uno | Costo del nivel |
|---|---|---|---|
| 0 | 1 | $n$ | $c\,n$ |
| 1 | 3 | $n/2$ | $(3/2)\,c\,n$ |
| 2 | 9 | $n/4$ | $(3/2)^2\,c\,n$ |
| $i$ | $3^i$ | $n/2^i$ | $(3/2)^i\,c\,n$ |
| $\log_2 n$ (hojas) | $3^{\log_2 n} = n^{\log_2 3}$ | 1 | $\Theta(n^{\log_2 3})$ |

Cada nodo tiene 3 hijos, así que el nivel $i$ tiene $3^i$ nodos de tamaño $n/2^i$; su costo es $3^i \cdot c\,n/2^i = (3/2)^i c\,n$, que *crece* hacia abajo porque los nodos se triplican pero su tamaño solo se reduce a la mitad. La altura es $\log_2 n$. Sumando la serie geométrica de razón $3/2$ más las hojas, y usando $(3/2)^{\log_2 n} = n^{\log_2 3}/n$:

$$\begin{aligned} T(n) &= \sum_{i=0}^{\log_2 n - 1} \left(\tfrac{3}{2}\right)^i c\,n + \Theta\!\left(n^{\log_2 3}\right) = 2c\,n\left[\left(\tfrac{3}{2}\right)^{\log_2 n} - 1\right] + \Theta\!\left(n^{\log_2 3}\right) \\ &= 2c\left(n^{\log_2 3} - n\right) + \Theta\!\left(n^{\log_2 3}\right) = \Theta\!\left(n^{\log_2 3}\right). \end{aligned}$$

**Por qué el exponente es $\log_2 3$.** Sale de contar hojas: altura $\log_2 n$ y 3 hijos por nodo dan $3^{\log_2 n} = n^{\log_2 3}$ casos base. El 3 es el número de productos y el 2 cuánto se reduce el tamaño; con 4 productos serían $4^{\log_2 n} = n^2$ hojas. **Lo que aporta** frente al Teorema Maestro es ver la estructura: el costo aumenta nivel a nivel y por eso domina el último.

## 4. Implementación y validación

- **Representación:** `std::string` de dígitos, el más significativo a la izquierda. Ningún tipo nativo guarda el producto completo; los `int` operan dígito a dígito.
- **Módulos:** `operaciones` (sumar, restar, desplazar, partir, normalizar, rellenar), `algoritmos` (`multiplicacionClasica`, `karatsuba`) y `main` (menú, validación y medición).
- **Relleno:** `karatsuba` normaliza y rellena una sola vez a la potencia de 2 $\geq$ la longitud mayor. Dentro de la recursión solo se igualan las longitudes de los operandos, porque $x_1 + x_0$ puede tener $m + 1$ dígitos; `partir` usa $m = \lceil n/2 \rceil$.
- **Caso base (`UMBRAL`):** si $n \leq$ `UMBRAL` se usa la clásica. El mínimo es 3: con $n = 2$ o $n = 3$, $x_1 + x_0$ puede tener $n$ dígitos (por ejemplo $9 + 9 = 18$ o $9 + 99 = 108$), así que $P_3$ tiene el mismo tamaño que la llamada que lo genera y la recursión no termina. Desde $n = 4$, $x_1 + x_0$ tiene a lo sumo $m + 1 < n$ dígitos. `UMBRAL` es una variable global (64 por defecto) que la medición cambia a 3, 32 y 64.

**Validación.** La clásica se verifica contra valores conocidos ($99 \times 99$ y $123456789 \times 987654321$). Luego Karatsuba se compara contra la clásica en 14 casos, con `UMBRAL` $= 3$ (para que hasta los casos pequeños pasen por $P_1, P_2, P_3$, la resta y la recomposición) y con `UMBRAL` $= 64$: un dígito, acarreos, multiplicación por cero, ceros a la izquierda (hasta 200), ceros intercalados, potencias de 10, longitudes impares y distintas, y aleatorios de $300 \times 300$ y $517 \times 333$ dígitos. Los 30 chequeos dan OK.

## 5. Análisis experimental

**Montaje.** Lenovo Legion, Intel Core Ultra 9 275HX; Ubuntu 26.04 LTS; g++ 15.2.0 con `-std=c++17 -O2`. Reloj `std::chrono::steady_clock`; cada multiplicación se repite duplicando $k$ hasta superar 10 ms y se reporta total/$k$, tras una llamada de calentamiento. Entradas aleatorias generadas fuera del cronómetro con semilla `srand(42 + c)` (la corrida $c$ usa los mismos operandos en los tres umbrales).

El largo de cada resultado se acumula y se imprime para que `-O2` no elimine el cálculo. La opción 2 del programa hace 3 corridas por `UMBRAL` y guarda las 9 tablas en `resultados/`; reportamos la mediana.

**Tabla 1:** Tiempo por multiplicación ($\mu$s), mediana de 3 corridas. La clásica es la mediana de las 9 corridas.

| $n$ | Clásica | Karatsuba U=3 | Karatsuba U=32 | Karatsuba U=64 |
|---|---|---|---|---|
| 2 | 0,061 | 0,125 | 0,127 | 0,126 |
| 4 | 0,163 | 0,855 | 0,191 | 0,192 |
| 8 | 0,285 | 3,11 | 0,334 | 0,337 |
| 16 | 0,832 | 10,7 | 0,911 | 0,904 |
| 32 | 3,29 | 33,0 | 3,15 | 3,13 |
| 64 | 13,5 | 109 | 13,8 | 12,9 |
| 128 | 54,5 | 335 | 45,8 | **42,9** |
| 256 | 233 | 999 | 145 | 146 |
| 512 | 935 | 3 037 | 461 | 425 |
| 1024 | 3 502 | 9 264 | 1 407 | 1 318 |
| 2048 | 13 626 | 27 462 | 4 294 | 4 092 |
| 4096 | 54 426 | 81 263 | 12 674 | 12 380 |

**El crecimiento coincide con la teoría.** Al duplicar $n$, el clásico multiplica su tiempo por 3,7–4,3 desde $n = 256$ (teórico $2^2 = 4$) y Karatsuba `U=3` por 2,96–3,05 (teórico $2^{\log_2 3} = 3$). Ajustando una recta en log-log entre 256 y 4096, las pendientes son **1,96** para la clásica y **1,59** para Karatsuba `U=3` (1,61 con `U=64`): coinciden con 2 y con $\log_2 3 \approx 1{,}585$.

**Punto de cruce.** Con `UMBRAL` $= 64$, Karatsuba gana desde $n = 128$ y en $n = 4096$ tarda 12,4 ms contra 54,4 ms (4,4 veces más rápido). En $n = 32$ y 64 ambos ejecutan lo mismo ($n \leq$ `UMBRAL`) y el cociente es $\approx 1$. En $n = 128$ hay un solo nivel de división: tres productos clásicos de 64 dígitos ($3 \times 13{,}5 \approx 40\,\mu$s) más el trabajo lineal dan 42,9 $\mu$s, cerca del 3/4 teórico de los 54,5 $\mu$s del clásico. Con `UMBRAL` $= 3$ el cruce **no aparece** hasta 4096; en una corrida extra hasta 65 536 dígitos el cociente pasa de 1,27 en $n = 16\,384$ a 0,76 en $n = 32\,768$, así que el cruce está entre esos tamaños.

**Por qué el clásico gana con números pequeños.** La complejidad asintótica oculta las constantes, y las de nuestro Karatsuba son grandes: cada llamada crea más de 15 cadenas nuevas (rellenar, cuatro `substr` en `partir`, sumas y restas con su `invertir` y `normalizar`, desplazamientos y recomposición), cada una con reserva de memoria en el heap y copia de dígitos, además del costo de tres llamadas recursivas. El clásico reserva un solo `vector<int>` y recorre un doble ciclo que el compilador optimiza bien y que aprovecha la caché. Con `UMBRAL` $= 3$ eso hace a Karatsuba hasta 12,9 veces más lento en $n = 16$; la ventaja asintótica de 4/3 por cada duplicación reduce esa desventaja (10,9 $\to$ 3,25 $\to$ 1,49 en $n = 8$, 512 y 4096) pero no la compensa antes de 4096. El `UMBRAL` resuelve el problema: los subproblemas pequeños, donde Karatsuba pierde, van al clásico.

## 6. Conclusiones y dificultades

- Pasar de cuatro a tres productos por nivel reduce las hojas del árbol de $n^2$ a $n^{\log_2 3}$, y por eso el orden baja de $\Theta(n^2)$ a $\Theta(n^{1{,}585})$. El Teorema Maestro (caso 1) y el árbol llegan al mismo resultado.
- Las mediciones lo confirman: pendientes de 1,96 y 1,59 en escala log-log.
- Mejor complejidad no significa más rápido para todo $n$: sin caso base grande, las copias y reservas de memoria hacen que Karatsuba solo gane por encima de unos 16 000–32 000 dígitos. Con `UMBRAL` $= 64$ el cruce baja a 128 dígitos y la ventaja crece con $n$.

**Dificultades.** (1) La recursión no terminaba con caso base menor que 3, porque $P_3$ no se achica; lo resolvimos con `UMBRAL` $\geq 3$. (2) El relleno a potencia de 2 solo sirve para la primera partición: las sumas intermedias rompen la longitud, así que igualamos longitudes en cada llamada. (3) La resta sin signo

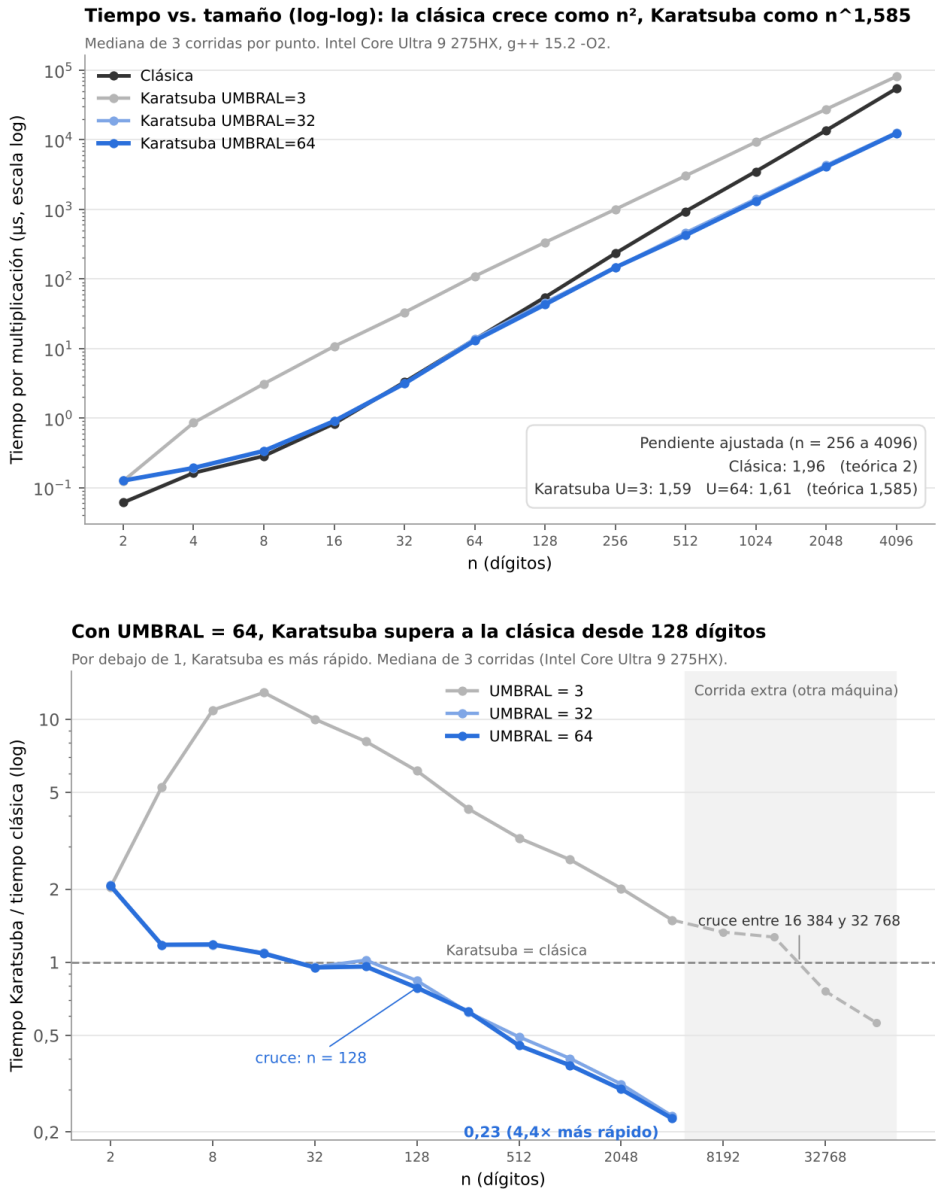



**Figura 1:** Arriba: tiempo vs. tamaño en escala log-log. Abajo: tiempo de Karatsuba dividido por el del clásico (debajo de 1, Karatsuba gana); la zona gris es una corrida extra de `UMBRAL` = 3 en otra máquina (Intel Xeon 2,1 GHz).

exige el orden $(P_3 - P_1) - P_2$ y aceptar sustraendos con ceros a la izquierda. (4) Para $n$ pequeño una multiplicación dura menos de 1 $\mu$s, por eso repetimos hasta 10 ms y consumimos el resultado.

## Declaración de uso de Inteligencia Artificial

Usamos Claude (Anthropic) como apoyo para entender el funcionamiento del código y corregir errores, y también como apoyo en la organización de la tabla y las gráficas de tiempos y en la redacción de este informe. El equipo revisó todo el código y cada paso del análisis, y cada integrante puede explicarlos en la defensa individual.